Code review — spring_initializare

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 3 — 2026-10-27 — sha 2f63483

Elev: Alex. Proiect: `cafenea-spring/`. Revizuite: `5f4ea7a` (etapa 4), `2c26892` (etapa 5), merge `2f63483`.

Rezumat față de runda 2: proiectul compilează (`mvn -q -DskipTests compile` fără erori), iar partea nouă are lucruri bune: `@Primary` / `@Qualifier` pe cele două bon-uri, `Reducere` ca interfață cu două implementări, `@Scope("prototype")` + `ObjectProvider` pentru `Comanda`, `@Lazy` pe raport, `@DependsOn` pe repository. Etapa 5 se comportă ca în cerință: `[Jurnal] pornit` apare înaintea `[Produse] repository creat`, `[Raport] creat acum` apare abia la opțiunea 5, iar raportul arată `Instante ComandaRepository: 1`. Dar toate cele 4 🔴 din runda 2 au rămas neatinse (B1–B4), iar etapa 4 are două probleme noi: ordinea reducerilor (B5) și bon-ul simplu care nu e folosit (B6). B1 e cel care blochează: cât timp meniul nu e cel din vanilla, scenariile din CERINTE nu se pot rula așa cum sunt scrise, deci nici etapele 4 și 5 nu se pot verifica cu `diff`.

Verificat prin rulare (pe o copie, nu în repo-ul tău; soluția profesorului rulată cu același input):

Verificare	Rezultat
`mvn -q -DskipTests compile`	fără erori
Etapa 1: `2,2,2,4,3,4,5,0`	`NoSuchElementException` la `ViewCafenea.java:58`; vanilla: bon cu `TOTAL 26.62 lei`
Etapa 4: `2,2,2,3,2,4,6,0` pe meniul tău	aceeași excepție (opțiunea 2 consumă tot inputul ca id-uri)
Etapa 4, aceeași comandă (Cappuccino + Latte + Croissant) introdusă pe meniul tău: `2,2,3,4,0,6,0`	`TOTAL 37.81 lei`; soluția: `TOTAL 37.93 lei`
Etapa 5: `5,0`	`[Jurnal]` înainte de `[Produse]` ✅, `[Raport] creat acum` abia la `5` ✅, `Instante ComandaRepository: 1` ✅
`--cafenea.tva=0.19 --cafenea.moneda=EUR --cafenea.nume=Zzz`	linia de debug `nume=Zzz, moneda=EUR`; antetul rămâne `CAFENEA`, bonul `TOTAL 23.80 lei`
`--spring.config.location=optional:classpath:/nu-exista.yml`	contextul nu pornește (`calculatorPret` nu găsește `${cafenea.tva}`)
`mvn test`	`Unable to find a @SpringBootConfiguration`

🔴 Critice

B1 🔴 Meniul tot nu e cel din vanilla: scenariile din cerință se opresc cu excepție (redeschis a doua oară)

`ViewCafenea.java:66`

CUM E ACUM: opțiunea 2 e tot „Comanda noua", cu bucla `do/while` care cere id-uri până la `0` (`:74–86`) și `comandaCurenta = comandaService.comandaNoua()` la fiecare „2" (`:67`). Scenariul `2,2,2,4,3,4,5,0` nu se termină: produsele `2,2,4,3,4` sunt adăugate pe rând, `5` primește „Nu exista acest produs", `0` închide bucla, iar `scanner.nextInt()` de la `:58` nu mai are ce citi. Vanilla, același input: bon cu `TOTAL 26.62 lei`.

DE CE: în vanilla, opțiunea 2 înseamnă „adaugă UN produs"; fiecare rând de input e o decizie a utilizatorului. La tine, după un „2" tot ce urmează e citit ca id de produs până la primul `0`, deci cele 8 rânduri ale scenariului sunt interpretate cu totul altfel decât le interpretează vanilla. Când stdin se termină, `Scanner.nextInt()` nu așteaptă, aruncă `NoSuchElementException`, și vezi stiva de la `ViewCafenea.java:58`. Etapele 1–5 sunt gândite ca să dovedească că Spring a schimbat doar cine creează obiectele, nu ce face programul; dovada e `diff`-ul de ieșire față de vanilla, iar `diff`-ul nu are sens cu un alt meniu. De aceea B1 blochează și verificarea etapelor 4 și 5.

FIX: copiază `ViewCafenea` din `cafenea-vanilla-java/src/app/view/` și schimbă doar dependențele (vin din constructor) și, în etapele următoare, ce cere fiecare etapă (antet, opțiunea 6). Opțiunea 2 = o singură cerere de id adăugată la comanda curentă; comanda se creează o dată la start; după `4` se creează alta.

Repro: `printf '2\n2\n2\n4\n3\n4\n5\n0\n' | java -jar target/cafenea-spring-0.0.1-SNAPSHOT.jar` -> `java.util.NoSuchElementException at mycode.products.view.ViewCafenea.play(ViewCafenea.java:58)`.

Același bug la (repară-le pe toate): - `ViewCafenea.java:67`: `comandaNoua()` la fiecare „2" (produsele din comanda în lucru se pierd). - `ViewCafenea.java:71-86`: bucla cu sentinelă `0`, promptul „ID produs: " repetat și „Introdu Id-ul produsului:". - `ViewCafenea.java:113`: `comandaCurenta = null` după finalizare; vanilla întoarce o comandă nouă. Efect verificat: `4,3,0` pe aplicația proaspătă dă „Nu exista o comanda activa." de două ori, în vanilla comanda există mereu. - `ViewCafenea.java:90-93`, `:106-109`, `:122-125`: ramurile „Nu exista o comanda activa." (aceeași cauză ca mai sus). - `ViewCafenea.java:105-114`: lipsește verificarea `getProduse().isEmpty()` înainte de finalizare („Comanda e goala"). - `ViewCafenea.java:58`, `:76`: `scanner.nextInt()` în loc de `Integer.parseInt(scanner.nextLine().trim())`. - `ViewCafenea.java:48-56`: textele meniului („========== CAFENEA ==========", „Afiseaza produse", „Comanda noua", „Alege optiunea:"), `:81-83` („Produs adaugat cu succes!", „Nu exista acest produs"), formatul bonului `:97-102` și al listei `:117`; toate diferă de vanilla. - `ViewCafenea.java:46`: `println(cafeneaProperties)` (vezi B3).

💡 **Hint:** pornești de la fișierul din vanilla și schimbi doar ce cere etapa. Primul pas măsurabil: aceleași 8 rânduri în ambele aplicații, `diff` pe ieșiri, fără rânduri diferite în afara liniilor `[...]` din etapă.

B2 🔴 `@PropertySource` și `program.properties` tot lipsesc (neatins)

`AppConfig.java:10`

CUM E ACUM: `src/main/resources/` conține doar `application.yml`, cu `program : 08:00-22:00` (`:13`). `PropertySource` e importat la `AppConfig.java:10` și nefolosit. Cerința de etapă 3: `program.properties` cu `cafenea.program=08:00-20:00`, încărcat cu `@PropertySource` pe `AppConfig`.

DE CE: Boot încarcă singur `application.yml` / `.properties`. Un fișier cu alt nume nu există pentru Spring până nu i-l arăți: `@PropertySource` îl adaugă în `Environment`, ca încă o sursă de chei. Exercițiul voia exact acest lucru, o a doua sursă pe care Boot n-o vede singur. Fără ea, toate cheile vin din yml, iar adnotarea nu a lucrat niciodată în proiectul tău. Proba din tabel: cu yml-ul scos, contextul nu mai pornește și nu există nicio altă sursă care să dea măcar `program`.

FIX: `src/main/resources/program.properties` cu `cafenea.program=08:00-20:00`; `@PropertySource("classpath:program.properties")` pe `AppConfig`; scoate `program` din `application.yml`.

Repro: `ls src/main/resources` -> `application.yml`.

Același bug la: - `AppConfig.java:30-35`: varianta comentată cu `@Value("${cafenea.nume/moneda/program}")`. - `application.yml:13`: `program` încă definit în yml. - `AppConfig.java:26`: `@Value("${cafenea.tva}")` pe parametrul unui `@Bean` (vezi M3).

B3 🔴 `CafeneaProperties` e injectat, dar antetul și moneda nu vin din el (neatins)

`ViewCafenea.java:48`

CUM E ACUM: singura folosire a recordului e `System.out.println(cafeneaProperties)` (`:46`). Antetul e „========== CAFENEA ==========" (`:48`), „lei" e scris de mână la `:102` și `:117` (și în `Produs.java:28`, `DetaliatBonPrinter.java:38`). `application.yml:10` are `nume: test`, deci nici după ce folosești `nume()` nu ar ieși „Cafenea". Cerința: `=== Cafenea · 08:00-20:00 ===`.

DE CE: `@ConfigurationProperties` doar umple un obiect: leagă `cafenea.nume` -> `nume`, `cafenea.moneda` -> `moneda` la pornire. Obiectul nu schimbă nimic singur; codul care afișează trebuie să citească din el. Proba din tabel: cu `--cafenea.moneda=EUR` recordul are `moneda=EUR`, dar bonul scrie `TOTAL 23.80 lei`, pentru că „lei" e text în `printf`. Lanțul yml -> `Environment` -> record merge; ultimul capăt, record -> ecran, lipsește.

FIX: `System.out.printf("%n=== %s · %s ===%n", cafeneaProperties.nume(), cafeneaProperties.program());`; `moneda()` în loc de „lei" în toate locurile de mai jos; șterge `println`-ul de la `:46`; `nume: Cafenea` în yml; câmpul `final`.

Repro: `printf '2\n2\n4\n0\n3\n0\n' | java -jar target/cafenea-spring-0.0.1-SNAPSHOT.jar --cafenea.moneda=EUR` -> `CafeneaProperties[nume=test, moneda=EUR, ...]`, antet `CAFENEA`, `TOTAL 24.20 lei`.

Același bug la: - `ViewCafenea.java:48`: antet hardcodat; `:102`, `:117`: „lei" hardcodat. - `DetaliatBonPrinter.java:38`: „lei" hardcodat (nou, etapa 4) și `:36`: „TVA 21%%" scris în text, în loc de procentul din configurare. - `Produs.java:28`: `toString` cu „lei" (modelul nu are acces la proprietăți; formatează în view). - `ViewCafenea.java:100-101`: `Subtotal` și `TVA` fără monedă. - `ViewCafenea.java:22`: `private CafeneaProperties` fără `final`.

B4 🔴 `@Autowired` pe câmp în `ProdusServiceImpl` (neatins)

`ProdusServiceImpl.java:14`

CUM E ACUM: câmpul `private ProdusRepository produsRepository` are `@Autowired` și nu e `final`; constructorul nu există. `new ProdusServiceImpl().listProduse()` compilează și cade cu `NullPointerException`.

DE CE: cu `@Autowired` pe câmp, Spring creează întâi obiectul cu constructorul implicit (câmpul e `null`) și abia apoi pune dependența prin reflexie. În aplicație nu se vede, pentru că Spring face ambii pași; oricine face `new` (un test, alt programator) primește un obiect pe jumătate gata. Cu un singur constructor cu parametri, Spring îl folosește singur, fără `@Autowired`, iar câmpul poate fi `final`: obiectul nu poate exista fără dependență. `@Autowired` are un rol real abia în etapa 4, pe CONSTRUCTORUL complet din `ViewCafenea` (vezi M5).

FIX: `private final ProdusRepository produsRepository;` + constructor cu parametru, fără `@Autowired`. Scoate importurile `Autowired` și `Component`.

Repro: (runda 2) `new ProdusServiceImpl().listProduse()` -> `NullPointerException ... "this.produsRepository" is null`. Codul clasei nu s-a schimbat de atunci.

Același bug la: nu există alt `@Autowired` în repo (`grep -rn Autowired src`: doar `ProdusServiceImpl.java:5` și `:14`). Dar `@Autowired` lipsește de unde ar fi trebuit (M5).

B5 🔴 Reducerile nu au `@Order`: se aplică în ordinea greșită, `TOTAL 37.81` în loc de `37.93` (nou, etapa 4)

`ReducereMeniu.java:7`, `ReducereFidelitate.java:6`

CUM E ACUM: `CalculatorPret` primește `List<Reducere>` (corect), dar nicio reducere nu are `@Order`. Pentru comanda din cerință (Cappuccino + Latte + Croissant), bonul detaliat dă `TOTAL 37.81 lei`; cerința: `37.93`, iar `37.81` e exact valoarea pentru ordinea inversată (fidelitate întâi, meniu după).

DE CE: Spring injectează un `List<Reducere>` în ordinea în care a găsit bean-urile, iar la scanare ordinea e alfabetică după numele clasei: `ReducereFidelitate` vine înaintea lui `ReducereMeniu`. Ordinea nu apare nicăieri în cod, deci ți-a ieșit „din întâmplare" cea greșită. Contează, pentru că reducerile nu comută: `35 × 0.95 − 2 = 31.25`, dar `(35 − 2) × 0.95 = 31.35`; după TVA, `37.81` față de `37.93`. `@Order(n)` e semnalul pe care Spring îl citește la injectarea unei liste: valoare mică = mai devreme în listă.

FIX: `@Order(1)` pe `ReducereMeniu`, `@Order(2)` pe `ReducereFidelitate` (`org.springframework.core.annotation.Order`).

Repro: `printf '2\n2\n3\n4\n0\n6\n0\n' | java -jar target/cafenea-spring-0.0.1-SNAPSHOT.jar` -> `TOTAL 37.81 lei`. Soluția, cu `2,2,2,3,2,4,6,0`: `TOTAL 37.93 lei`.

Același bug la: `grep -rn "@Order" src` -> zero rezultate; ambele clase de reduceri îl au de adăugat. `CalculatorPret.java:25` iterează lista în ordinea primită, deci nu e nimic de schimbat acolo.

B6 🔴 Bon-ul simplu nu e folosit niciodată, iar bon-ul detaliat nu arată reducerile (nou, etapa 4)

`ViewCafenea.java:89`, `DetaliatBonPrinter.java:32`

CUM E ACUM: `ViewCafenea` primește `BonPrinter bonPrinter` (rezolvat prin `@Primary` la `SimpluBonPrinter`) și îl memorează la `:37`, dar nu îl apelează nicăieri (`grep -n "bonPrinter\b"` -> doar

`:23`, `:31`, `:37`). Opțiunea 3 își scrie bonul inline la `:95–102`. `SimpluBonPrinter.printeaza` nu scrie `TOTAL`, deși cerința: „produsele și TOTAL". În `DetaliatBonPrinter`, cerința e „fiecare reducere aplicată", cu numele ei; la tine apare o singură linie `Total reduceri -3.75`, iar `Reducere.nume()` nu e apelat nicăieri în proiect.

DE CE: `@Primary` spune „dacă cineva cere un `BonPrinter` fără alte precizări, ia-l pe acesta". Alegerea are efect doar dacă există un loc care cere un `BonPrinter` și îl folosește. La tine alegerea se face corect (Spring nu se plânge de ambiguitate), dar rezultatul ajunge într-un câmp pe care nu-l citește nimeni; ieșirea nu se schimbă cu sau fără `@Primary`, deci nu ai cum vedea adnotarea lucrând. La fel `nume()`: o metodă din interfață pe care nu o cheamă nimeni nu poate ajunge pe bon.

FIX: `case 3` apelează `bonPrinter.printeaza(comandaCurenta)`; `SimpluBonPrinter` scrie produsele și `TOTAL` (prin `ComandaService`); `DetaliatBonPrinter` primește `List<Reducere>` sau diferența per reducere de la `ComandaService`, și scrie `reducere.nume()` cu valoarea ei (`Meniu cafea + croissant -2.00`, `Fidelitate -5% peste 30 -1.65`); `TVA` cu procentul din configurare.

Repro: `printf '2\n2\n3\n4\n0\n6\n0\n' | java -jar ...` -> `Subtotal 35.00`, `Total reduceri -3.75`, `TVA 21% 6.56`, `TOTAL 37.81 lei`. Soluția: două linii de reducere cu nume, `TVA 21% 6.58`, `TOTAL 37.93 lei`.

Același bug la: - `ViewCafenea.java:24`, `:126`: `bonDetaliatPrinter` e folosit (corect); doar `bonPrinter` e orfan. - `DetaliatBonPrinter.java:36`, `:38`: `21%` și „lei" scrise în text (vezi B3). - `DetaliatBonPrinter.java:4`: import `ComandaRepository` neutilizat.

🟡 Importante

M1 🟡 **`mvn test` pică** (din runda 1, neatins). `CafeneaSpringApplicationTests`: `Unable to find a @SpringBootConfiguration`; clasa principală e în `mycode.pornire`, sub pachetul testului.

M2 🟡 **`@Import(AppConfig.class)` de prisos** (din runda 1, neatins). `CafeneaSpringApplication.java:11`: `AppConfig` e în `mycode.config`, deja în `@ComponentScan("mycode")`.

M3 🟡 **`CalculatorPret` tot nu e `@Component`** (neatins). `AppConfig.java:24–28`: `@Bean` cu `@Value` pe parametru și `List<Reducere>`. Cerința: `@Component` pe `CalculatorPret`, `@Value` în constructorul lui; în `AppConfig` rămâne un singur `@Bean` (`Scanner`), la tine sunt 3.

M4 🟡 **Etapa 4 în pachetele proprii** (parțial închis). Interfețele `BonPrinter` și `Reducere` și cele patru implementări există acum ✅. Reducerile stau în `orders.service`, nu într-un pachet propriu `reduceri`; bon-urile sunt în `orders.printer` (ok).

M5 🟡 **`ViewCafenea` n-are al doilea constructor, deci nu există niciun `@Autowired` pe constructor.** `ViewCafenea.java:29`: cerința etapei 4 cere un al doilea constructor fără bonul detaliat; cu doi constructori Spring nu mai alege singur, deci `@Autowired` pe cel complet e exact adnotarea nouă a etapei. Acum există un singur constructor și adnotarea nu e exersată.

M6 🟡 **Numele bean-urilor din cerință.** `SimpluBonPrinter` / `DetaliatBonPrinter` trebuiau să se numească `simplu` și `detaliat`; la tine `@Qualifier("bondetaliat")` pe clasă (`DetaliatBonPrinter.java:11`) și `@Qualifier("bondetaliat")` pe parametrul din `ViewCafenea.java:31`; funcționează (aceeași valoare în ambele locuri), dar numele nu e cel cerut.

M7 🟡 **`RaportVanzari.afiseaza()` nu conține lista comenzilor.** `RaportVanzari.java:13–16` scrie „Raport vanzari" și numărul de instanțe; cerința: `afiseaza()` = lista de la opțiunea 5 + `Instante ComandaRepository: N`. Lista de la opțiunea 5 stă tot în `ViewCafenea.java:117`. Metoda statică se numește `getInstance()` (`ComandaRepositoryImpl.java:36`), nu `getInstante()`.

🟢 Cleanups

- **C1** importuri nefolosite: `Component` în `ProdusRepositoryImpl.java:5`, `ComandaRepositoryImpl.java:5`, `ProdusServiceImpl.java:6`, `ComandaServiceImpl.java:8`; `PropertySource` în `AppConfig.java:10`; `Configuration` în `CafeneaProperties.java:4`; `ComandaRepository` în `DetaliatBonPrinter.java:4`.
- **C2** cod comentat în `AppConfig.java:30–35`.
- **C3** `application.yml:13`: `program :` cu spațiu înainte de `:`.
- **C4** `ViewCafenea` în `products.view`, deși lucrează și cu comenzile.
- **C5** formatare: `private CafeneaProperties`, `String nume ,String moneda`, `"6.Bon detaliat"` (fără spațiu după punct, `ViewCafenea.java:54`), `implements BonPrinter{` (`SimpluBonPrinter.java:10`), `printeaza (Comanda` (`BonPrinter.java:6`).
- **C6** `ProdusRepositoryImpl.java`, `ComandaRepositoryImpl.java`, `ViewCafenea.java` nu au newline la final.

Before / After

Înainte	După (indicativ, în document, nu în codul tău)
`ViewCafenea` : `case 2:` cu `comandaNoua()` și buclă `do/while` până la `0`	`case 2 ->` o cerere de id adăugată la comanda curentă; `case 4` pe comandă goală: „Comanda e goala”; după finalizare `comandaCurenta = comandaService.comandaNoua()`
`@Component` `public class ReducereMeniu implements Reducere`	`@Component` `@Order(1)` `public class ReducereMeniu implements Reducere` (`@Order(2)` pe fidelitate)
`case 3:` bon scris inline în `ViewCafenea`	`case 3 -> bonPrinter.printeaza(comandaCurenta);`
`System.out.println(cafeneaProperties);` + `"========== CAFENEA =========="` + `"... lei"`	`printf("%n=== %s · %s ===%n", cafeneaProperties.nume(), cafeneaProperties.program());` + `cafeneaProperties.moneda()` în loc de „lei”
`@Autowired` `private ProdusRepository produsRepository;`	`private final ProdusRepository produsRepository;` + constructor cu parametru, fără `@Autowired`
`application.yml` cu `program` ; fără `program.properties`	`program.properties` cu `cafenea.program=08:00-20:00` ; `@PropertySource("classpath:program.properties")` pe `AppConfig`

Q&A

1. Reducerea de meniu (−2) și cea de fidelitate (×0.95) dau 31.25 într-o ordine și 31.35 în cealaltă. Cine decide ordinea în care ajung reducerile în `List<Reducere>` , și ce adnotare o fixează?
2. Ai pus `@Primary` pe `SimpluBonPrinter` . Dacă ștergi `@Primary` , ce se schimbă la pornire? Dar în ieșirea opțiunii 3, așa cum e acum codul? Ce ți-arată asta despre unde lucrează adnotarea?
3. În `ProdusServiceImpl` , ce se întâmplă, pas cu pas, între crearea obiectului și momentul în care câmpul cu `@Autowired` primește valoare? De ce nu există acest moment când dependența vine prin constructor?
4. În ce ordine apar `[Jurnal]` și `[Produse]` și ce adnotare o garantează? Ce s-ar întâmpla fără ea?

Runda 2 — 2026-10-23 — sha 3cb2bc8

- 🔴 B1 meniul ≠ vanilla: NEATINS, `NoSuchElementException` pe `2,2,2,4,3,4,5,0` (acum `ViewCafenea.java:58`), vezi B1 runda 3
- 🔴 B2 `@PropertySource` + `program.properties`: NEATINS, vezi B2 runda 3
- 🔴 B3 `CafeneaProperties` nefolosit (antet + monedă): NEATINS, `--cafenea.moneda=EUR` tot `lei`, vezi B3 runda 3
- 🔴 B4 `@Autowired` pe câmp în `ProdusServiceImpl`: NEATINS, vezi B4 runda 3
- 🟡 M1 `mvn test`: NEATINS · M2 `@Import`: NEATINS · M3 `CalculatorPret` ca `@Component`: NEATINS (tot 3 `@Bean`)
- 🟡 M4 etapa 4 abia începută: ÎNCHIS în fond (clasele și interfețele există, vezi M4 runda 3 pentru pachete)
- 🟢 C1 importuri: NEATINS (+3 noi) · C2 cod comentat: NEATINS · C3 `program :`: NEATINS · C4 pachet view: NEATINS · C5 formatare: NEATINS

Runda 1 — 2026-10-22 — sha 57e0657

- 🔴 B1 meniul ≠ vanilla: NEATINS. Același `NoSuchElementException` pe `2,2,2,4,3,4,5,0` (acum la `ViewCafenea.java:41`), vezi B1 runda 2
- 🟡 B2 `@PropertySource` pe yml: PARȚIAL. Adnotarea greșită e scoasă, dar `program.properties` și `@PropertySource` lipsesc, vezi B2 runda 2
- 🟡 B3 etapa 3 pe jumătate: PARȚIAL. ✅ TVA din yml (`--cafenea.tva=0.19` -> `TOTAL 26.18`, verificat prin rulare); ✅ record `CafeneaProperties` + `@ConfigurationProperties` + `@EnableConfigurationProperties`; 🔴 antetul și moneda neaplicate (B3 runda 2); 🟡 `CalculatorPret` nu e `@Component`, `AppConfig` are tot 3 `@Bean` (M3 runda 2)
- 🟡 M1 `mvn test`: NEATINS (verificat, aceeași eroare) · M2 `@Import`: NEATINS
- 🟢 C1 importuri: NEATINS (+2 noi) · C2 `program :`: NEATINS (C3 runda 2) · C3 pachet view: NEATINS (C4 runda 2)